

INDUS VALLEY AI

A Domain-Locked AI Platform for Harappan Scholarship

Project Lead	Mohammed Majeed Khan
Organization	AI Hub, Mahindra University, Hyderabad, India
Category	Educational AI Web Application — Static, Zero-Cost, Privacy-First
Date	April 2026
Technology	Three.js · Gemini Nano · Canvas Vision · Vanilla JS · Python ReportLab
Access	Open browser — no server, no API key, no account required

This document is a complete technical and conceptual record of the Indus Valley AI project. It covers every file, every design decision, every technology layer — written plainly so that anyone from a professor to a first-year student can follow what was built and why.

Table of Contents

1	Project Overview
2	What Makes This Different
3	How to Run the Website
4	Architecture — The Big Picture
5	File-by-File Walkthrough
5.1	index.html — The Skeleton
5.2	styles.css — The Design System
5.3	knowledge-base.js — The Brain
5.4	app.js — The Interactions Engine
5.5	vision.js — The Eyes
5.6	gemini-nano.js — On-Device LLM
5.7	three-scene.js — Hero 3D Seal
5.8	three-routes.js — 3D Trade Routes Globe
5.9	data.js — Curated Dataset
6	The AI Chat System — Deep Dive
7	Canvas Vision — How Image Analysis Works
8	Gemini Nano — Free On-Device Intelligence
9	3D Graphics — Two WebGL Scenes
10	Design System & Visual Identity
11	Tech Stack Summary Table
12	What We Built vs What Others Do
13	Future Directions
14	Conclusion

1. Project Overview

Indus Valley AI is a fully browser-based, zero-cost, zero-backend educational platform dedicated entirely to the Indus Valley / Harappan Civilization (c. 3300–1300 BCE). The project was built from scratch as a flagship demonstration of what modern web technology can achieve without any server infrastructure, paid APIs, or cloud fees.

Everything runs inside the user's own browser tab. The AI chat answers questions from a hand-curated knowledge base of 49 scholarly topics. When available, an on-device language model (Gemini Nano, built into Chrome) silently enriches answers — entirely on the user's device, with no data leaving their machine. Two real-time 3D scenes render using WebGL. Image analysis runs on a pixel-level canvas algorithm with no external vision API.

The platform was designed for three audiences: (a) the general public curious about the Indus Civilization, (b) students doing research, and (c) professors and archaeologists who want a rigorous, academically honest reference tool. Every answer is domain-locked — the system refuses to answer questions outside the Indus/Harappan domain, keeping the experience focused and trustworthy.

At a Glance

Website type	Static HTML/CSS/JS — no server needed
Backend	None. Zero. Everything runs in the browser.
AI model	Hand-built KB (49 topics) + optional Gemini Nano on-device
3D engine	Three.js 0.160 — two full WebGL scenes
Vision	Canvas pixel analysis — no camera API, no vision API
Database	None — all data is in JS files loaded at page open
Cost to run	USD \$0.00 — host on GitHub Pages or any static host
Domain coverage	Indus Valley Civilization only — strict domain lock
Seals dataset	12 real Harappan seal photographs from published corpus
Lines of code	~7,000 lines across HTML, CSS, JS (all hand-written)

2. What Makes This Different

Most AI chatbots are general-purpose — they will answer questions about cooking, sports, and software in the same breath as archaeology. That makes them unreliable for scholarship. Indus Valley AI is the opposite: it is hard-locked to one domain and academically honest within it.

Feature	Indus Valley AI	Typical AI chatbot
Domain	Indus Civilization only	Everything — unfocused
API cost	\$0 forever	\$5–\$100+/month
Privacy	100% — nothing sent to any server	Data goes to cloud
3D visuals	Two live WebGL scenes	None
Image analysis	Client-side canvas heuristics	Requires paid vision API
On-device LLM	Gemini Nano (Chrome built-in)	Cloud model required
Academic honesty	Flags debates, cites scholars by name	May hallucinate
Hosting	Any static host (GitHub Pages = free)	Needs server + DB
Offline use	Yes — after first load	No

3. How to Run the Website

Because the site uses ES Modules (modern JavaScript imports), it must be served over HTTP — it cannot simply be opened as a file:// URL. The simplest method is Python's built-in web server, which is available on every Mac and most Linux machines.

Step-by-step instructions (macOS / Linux)

Step 1 — Open Terminal

Press Cmd+Space, type Terminal, press Enter.

Step 2 — Navigate to the folder

```
cd ~/Desktop/majeedkhanwebsite/indus-valley
```

Step 3 — Start the server

```
python3 -m http.server 8000
```

Step 4 — Open the browser

Open Chrome or Safari. Go to: <http://localhost:8000>

Step 5 — Stop the server

When done, go back to Terminal and press Ctrl+C.

Tip for Gemini Nano: The on-device LLM feature only works in Google Chrome 127 or newer, with the Gemini Nano flag enabled in `chrome://flags`. The rest of the site works perfectly in any modern browser — Firefox, Safari, Edge — without it.

4. Architecture — The Big Picture

The project follows a single-page application (SPA) pattern implemented entirely with vanilla JavaScript — no React, no Vue, no build system, no Node.js. This was a deliberate choice: zero dependencies means zero security vulnerabilities from third-party packages, and the project will still work correctly in 10 years without any updates.

Script Load Order

Scripts are loaded in a specific order inside index.html. Each script adds its module to the window object so later scripts can use it:

#	File	Exports	Role
1	knowledge-base.js	window.IVA_KB	The 49-topic scholarly knowledge base — must load first
2	data.js	window.IVA_DATA	Seals, cities, motifs, script glyphs dataset
3	vision.js	window.IVA_VISION	Canvas pixel analysis for image uploads
4	gemini-nano.js	window.IVA_NANO	On-device Gemini Nano detection and session
5	three-scene.js	module	Three.js hero 3D seal (ES module, own scope)
6	three-routes.js	module	Three.js trade routes globe (ES module)
7	app.js	IIFE	All UI interactions — reads everything above

Data Flow for a Chat Question

When a user types a question and presses Send, the following happens in sequence — all inside the browser, in under one second:

1. app.js receives the question text from the input field.
2. The domain gate checks: does the question contain any Indus-domain keywords? Does it match a hard out-of-scope pattern (bitcoin, recipe, etc.)?
3. If in-domain: bestTopic() scores every KB topic by summing keyword match lengths. The highest-scoring topic wins.
4. The winning topic's title, paragraphs, and source citation are rendered into the chat as an HTML message bubble.
5. Asynchronously: maybeEnrich() checks if Gemini Nano is ready. If yes, it sends the question and the first paragraph of the KB answer to the on-device model and appends a green enrichment block when the response arrives.

6. If the user uploaded an image: vision.js analyzed it first and routed to the most relevant KB topic before step 2.

5. File-by-File Walkthrough

The project lives in one folder: `indus-valley/`. Here is every file and what it does.

<code>index.html</code>	~34 KB
Main page — all 12 sections, HTML skeleton	
<code>styles.css</code>	~45 KB
Complete design system — 1,200+ lines of CSS	
<code>knowledge-base.js</code>	~67 KB
49 scholarly topics, domain gate, fallbacks	
<code>app.js</code>	~26 KB
All UI — cursor, scroll, chat, upload, paths	
<code>vision.js</code>	~8 KB
Canvas image classifier — no API	
<code>gemini-nano.js</code>	~5 KB
On-device LLM bridge — Chrome only	
<code>three-scene.js</code>	~11 KB
Three.js hero 3D seal animation	
<code>three-routes.js</code>	~9 KB
Three.js 3D trade routes globe	
<code>data.js</code>	~18 KB
Seals, cities, motifs, script glyphs	
<code>assets/seals/</code>	~2 MB
12 real Harappan seal photographs (JPEG)	
<code>assets/logo.png</code>	user
AI Hub logo (user-supplied)	

5.1 `index.html` — The Skeleton

The HTML file defines the visual structure of the entire page. It is a single long document with 12 named sections that the browser scrolls through. Every interactive element has a `data-*` attribute so that `app.js` can find it without relying on fragile class names.

- Hero — full-viewport 3D canvas (the animated seal)
- Ticker — scrolling gold text band with civilization facts
- About — four key pillars of the Indus civilization
- AI Chat — the main AI interaction panel (centerpiece)
- Pillars — six achievement cards (urban planning, trade, etc.)

- Seal Gallery — 12 real seal photographs with modal zoom
- Cities — 10 excavated city cards (click for detail)
- Trade Routes — second 3D WebGL scene (the globe)
- Timeline — visual chronology from 7000 BCE to present
- Motifs — 8 recurring iconographic symbols
- Script — animated Indus sign strip
- Research — six academic reference links

Key HTML features: importmap (tells the browser where to find Three.js from CDN), custom cursor divs (.cursor-dot, .cursor-ring), the spotlight gradient div, the scroll spine aside element with 10 named nodes, the drag-drop overlay, the hidden file input, the attachment preview box, and the Gemini Nano status badge.

5.2 styles.css — The Design System

The CSS file defines the complete visual language of the project. It uses CSS Custom Properties (variables) extensively so that the entire color palette can be changed from one block at the top.

Core design tokens:

Variable	Value	Purpose
--bg	#0F0F0F	Page background — near-black charcoal
--gold	#D4AF37	Primary accent — ancient gold
--gold-soft	#C2A878	Secondary accent — warm bronze
--terracotta	#B55530	Highlight accent — fired clay red
--serif	Cormorant Garamond	Display typeface — elegant, historical
--sans	Inter	Body typeface — clean, legible
--mono	JetBrains Mono	Code typeface — technical sections

Notable CSS techniques used:

- Custom cursor — two pseudo-elements with different lag speeds (mix-blend-mode: difference makes it invert on hover)
- Spotlight — a radial gradient whose center follows the mouse via CSS custom properties --mx and --my updated in JavaScript
- Scroll spine — a fixed left sidebar with 10 labeled nodes; a .spine-fill bar uses scaleY() to show scroll progress
- Reveal animations — IntersectionObserver adds .in class; CSS transitions fire with opacity + scale (cubic-bezier timing)
- Magnetic buttons — pointermove listener computes offset from center and applies a CSS transform

- Drop zone — dashed gold pulsing border overlay that appears on file drag-enter
- Glass cards — `backdrop-filter: blur()` for frosted-glass seal cards
- Full-bleed 3D canvas — `position:sticky` + `height:100vh` so the WebGL scene scrolls with the page correctly

5.3 knowledge-base.js — The Brain

This is the most important file — it is what makes the AI answers accurate and trustworthy. It contains 49 hand-curated scholarly topics, each written with care to reflect current archaeological consensus while flagging ongoing debates.

Structure of each topic:

Every topic is a JavaScript object with four fields: keys (array of trigger phrases for matching), title (displayed heading), body (array of paragraphs — the actual answer), and sources (citation string).

Example structure:

```
{ keys: ['great bath', 'ritual bathing', 'mohenjo-daro bath'], title: 'The Great Bath of Mohenjo-daro', body: ['First paragraph...', 'Second paragraph...'], sources: 'Marshall 1931; Jansen 1993' }
```

Coverage — all 49 topic categories:

- Civilization overview
- Chronology & phases
- Mohenjo-daro
- Harappa
- Dholavira
- Lothal — Bronze Age port
- Rakhigarhi
- Sutkagan-Dor
- Shortugai outpost
- Indus script — status
- Fish sign (most frequent)
- Parpola hypothesis
- Yajñadevam claim
- Seals — what they are
- Unicorn motif
- Seal manufacture
- Ernestite drill
- Terracotta figurines
- Priest-King statuette
- Dancing Girl bronze
- Daimabad bronzes
- Toys & games
- Brick technology
- Urban planning & drainage
- Trade with Mesopotamia
- Meluhha (Mesopotamian name)
- Carnelian bead exports
- Copper & bronze metallurgy
- Pottery typology
- Weights & measures
- Indus religion & fire altars
- Why no kings debate
- Burials & mortuary
- Decline of the civilization
- 4.2ka climate event
- Ghaggar-Hakra river
- Aryan migration context
- Rakhigarhi ancient DNA 2019
- Comparative: vs Mesopotamia
- Comparative: vs Egypt
- Archaeologist directory
- Motifs — pipal leaf
- Motifs — swastika
- Script glyphs & sign list
- Architecture
- Agriculture & diet
- Domain fallback
- Out-of-domain response

- Great Bath
- (5 additional topics covering debates & schools)

Domain gate — how out-of-scope questions are blocked:

Two layers of protection. First, an `inDomainKeywords` array of ~80 Indus-related terms is checked against the question. Second, a regex pattern `outOfScopeHard` matches obvious non-Indus topics (bitcoin, cricket score, recipe, GPT-4, etc.) and blocks them regardless of other matches. If both checks fail AND no KB topic scores above the minimum threshold, the system returns a polite out-of-scope response.

5.4 app.js — The Interactions Engine

app.js is the conductor — it initializes all interactive features and wires them together. It is written as a single IIFE (Immediately Invoked Function Expression) so that all variables are scoped privately and nothing pollutes the global window object.

Major systems inside app.js:

Custom Cursor

Two div elements (dot + ring) animate at different speeds using requestAnimationFrame. The dot follows the pointer at 30% speed; the ring follows at 12% — the lag creates a trailing effect. On touch devices (pointer:coarse), the cursor is hidden entirely.

Spotlight

The mouse position is tracked via pointermove and written to CSS custom properties --mx and --my. A CSS radial-gradient centered at (--mx, --my) creates the glow effect with no canvas involved.

Magnetic Buttons

On pointermove over certain buttons, the offset between the pointer and the button center is computed and applied as a CSS transform translation at 25% magnitude. The button snaps back when the pointer leaves.

Scroll Spine

An IntersectionObserver tracks which section is nearest to the viewport center. The matching spine node gets the .active class. Clicking a spine node scrolls smoothly to that section. The spine fill bar's scaleY tracks overall scroll progress.

Reveal on Scroll

A second IntersectionObserver watches all elements with the .reveal class. When they enter the viewport, the .in class is added and a CSS transition fires (opacity 0→1, scale 0.96→1, 800ms cubic-bezier easing).

File Upload

A hidden is triggered by a visible Attach button's click. On file selection, handleFile() asynchronously loads the file — images go to vision.js for pixel analysis, text files are read with FileReader. Drag-and-drop onto the window also triggers handleFile().

Learning Paths

Five curated learning sequences (Seals, Cities, Script, Trade, Decline) are pre-defined as ordered arrays of topic titles. Clicking a path button plays the topics in sequence with step badges and optional Gemini Nano enrichment.

Video Redirect

The Video Gen button constructs a Veo-style prompt string based on the last topic discussed, then opens Google AI Studio in a new tab with the prompt pre-filled in the URL. No API key needed — the user's own Gemini quota is used.

5.5 vision.js — The Eyes

vision.js provides image intelligence with zero API calls. When a user uploads an image, this module analyzes it on the client using HTML Canvas pixel data.

How it works (plain English):

1. The image is drawn onto a hidden 96×96 pixel canvas (tiny — fast to process).
2. The raw pixel RGBA values are read with `getImageData()`.
3. Five statistics are computed from the pixel data: average brightness (0–255), sepia warmth score (how brown/cream the image is, 0–1), edge density (how sharp the outlines are), dominant hue (red/green/brown/gray/cool), and aspect ratio (width÷height).
4. Edge density uses a lightweight Sobel approximation: for each pixel, compute the luminance gradient to the right and downward neighbor. Average all gradients, normalize to 0–1. Higher = more edges = more detailed / line-art like.
5. Six classification rules run in order: seal (warm+square+photo), script (line-art+light+mono), map (green/brown-green hue), artifact (warm+photo), illustration (light+low-color), site-photo (photo+dark). The first matching rule wins.
6. Each category maps to 2–3 suggested KB topic titles. The first title that exists in the KB is returned as the matched topic.
7. The result is displayed to the user with all the raw statistics visible.

Why not use a real computer vision API? Two reasons: (1) privacy — the user's image never leaves their device; (2) cost — zero. The heuristic approach works well for the expected inputs: seal photographs, script transcriptions, site maps, and figurine photos. For general photographs it gives a reasonable best-guess and falls back gracefully.

5.6 gemini-nano.js — On-Device LLM

Gemini Nano is a small language model built directly into Google Chrome (version 127+). It runs on the user's GPU/CPU — no internet connection is used. `gemini-nano.js` is the bridge between the website and this model.

What it does:

- `detect()` — Checks if the browser supports `window.LanguageModel` (new API) or `window.ai.languageModel` (legacy API). Sets a badge on the chat panel showing the state: ready, downloading, or unsupported.
- `ensureSession()` — Creates a language model session lazily (only when first needed). Passes a system prompt that locks Gemini Nano to the Indus domain with the same academic honesty rules as the KB.
- `ask(question, contextNote)` — Sends the user's question plus the first paragraph of the KB answer as context. Asks the model to add one additional concise paragraph that does not repeat the KB content. Returns the text or null if unavailable.

- The enrichment paragraph is shown in a green block labeled 'Gemini Nano · on-device' at the bottom of the bot message.

The system prompt inside `gemini-nano.js` enforces: domain restriction to Indus topics, academic honesty (flag debates, name scholars), correct statement that the Indus script is undeciphered, correct Rakhigarhi 2019 ancient-DNA finding, and conciseness (under 200 words).

5.7 three-scene.js — Hero 3D Seal

The hero section shows a 3D animated Indus-style stamp seal. It is rendered using Three.js, a JavaScript library that wraps WebGL (the browser's 3D graphics API). Everything is procedurally generated — no external 3D model files are loaded.

What is in the scene:

- Seal body — a rounded rectangular block made with `ExtrudeGeometry` of a rounded `Shape`. Material: `MeshStandardMaterial` with gold color, moderate metalness, and emissive glow.
- Unicorn bull — the most iconic Indus motif. Built from a bezier path using `THREE.Path` commands that approximate the animal's outline. Extruded to give it depth.
- Horn mesh — a separate tapered `CylinderGeometry` attached to the animal's head.
- Manger / standard — a short rectangular object below the animal's mouth, common on real Indus unicorn seals.
- Script signs — a row of 7 small extruded glyphs representing Indus script signs, placed along the top of the seal face.
- Floating rings — two animated torus rings orbit the seal to add life.
- Gold dust particles — 600 `Points` (sprites) drift upward and reset, creating a shimmering golden atmosphere.
- Pointer parallax — the camera subtly tilts toward the mouse position for a 3D depth illusion on desktop.
- `IntersectionObserver` — animation pauses automatically when the hero scrolls off screen to save GPU power.

5.8 three-routes.js — 3D Trade Routes Globe

A second Three.js scene shows the Indus trade network as glowing arcs over a stylized ancient world disc. Nine city nodes are placed at historically accurate relative positions.

Scene contents:

- Ground disc — a large `CircleGeometry` with concentric ring decorations, styled to look like an antique world map plate.
- City nodes — 9 sites (Indus, Sumer, Magan, Dilmun, Shortugai, Susa, Meluhha, Lothal, Dholavira) rendered as gold/terracotta pillars with glowing dot tops and pulsing halo rings.
- Trade route arcs — 8 `QuadraticBezierCurve3` paths connecting the cities, rendered as `TubeGeometry` meshes with semi-transparent gold material.
- Travelling sparks — one `SphereGeometry` per route that travels along the Bezier curve at random speeds, representing goods in transit.
- Ambient gold dust — 240 floating `Points` particles drift upward.
- Slow camera orbit — the camera revolves around the scene using $\sin/\cos$ of elapsed time, tilted by mouse position.

5.9 data.js — Curated Dataset

data.js provides three structured datasets that drive the visual sections of the page:

seals[]	12 real seal photographs — each with id, filename, label, and tag (unicorn, zebu bull, elephant, etc.)
cities[]	10 excavated cities — name, location, summary paragraph, stats chips (area, pop. estimate, key finds), and expanded detail bullet points
motifs[]	8 recurring motifs — name, frequency, summary, SVG glyph path string, and detail paragraphs
scriptGlyphs[]	16 SVG path strings representing stylized Indus signs for the animated script strip

6. The AI Chat System — Deep Dive

The chat panel is the centerpiece of the project. It looks and feels like a modern AI assistant (typing indicators, streaming feel, avatar, attachment previews) but runs entirely offline from a curated knowledge base.

Topic Scoring Algorithm

Each of the 49 KB topics has a `keys[]` array of trigger phrases. When a question arrives, every topic is scored by summing the lengths of all keys that appear in the question (lowercased). Using key length as the score weight is the key insight: longer, more specific phrases (like 'how was a seal actually made') score higher than short generic words (like 'seal'). This prevents common words from drowning out specific matches.

```
Example: question = 'how did they make the indus seals?' Topic A: keys=['seal','seals']  
→ score = 4 + 5 = 9 Topic B: keys=['how a seal was actually made','seal manufacture'] →  
score = 27 + 16 = 43 Winner: Topic B (seal manufacture)
```

Quick Questions Panel

Eight pre-set question buttons let users explore without typing. Each button has a `data-q` attribute with the full question text, which is passed directly to `handleQuestion()` as if the user had typed it.

Learning Paths

Five curated learning journeys walk users through a topic sequence step by step. Each step is delivered with a numbered badge (e.g. '2/5 · The unicorn motif'). Between steps, a 450ms pause is added so the user can read before the next card appears. Gemini Nano enrichment is attempted for each step.

Attachment + Vision Branch

When an image is attached, the chat handler takes a different path: vision analysis runs first and its category result is shown to the user with all raw statistics. Then the matched KB topic is displayed as a follow-up card. The user's uploaded image is shown as a thumbnail inside their message bubble.

7. Canvas Vision — How Image Analysis Works

Traditional image AI requires sending the image to a cloud API (Google Vision, AWS Rekognition, etc.) which costs money and involves privacy trade-offs. Indus Valley AI instead runs a statistical classification algorithm directly in the browser — no upload, no API key, zero latency beyond the pixel loop.

The Sepia Score (warmth metric)

Indus seals are made of steatite (soapstone) — a warm beige/brown stone. Photographs of seals tend to have a characteristic warm sepia tone. The sepia score captures this: it computes how much redder and warmer the average pixel is compared to a neutral gray, then scales the result between 0 and 1. Seals and clay artifacts typically score above 0.45.

```
Formula: warmth = (avgR - avgB) / 255 balance = 1 - |((avgR-avgG) - (avgG-avgB))| / 255
sepia = clamp(warmth * 1.6 * balance, 0, 1)
```

Edge Density (Sobel-light approximation)

Edge density measures how much fine detail and sharp outlines are in the image. High edge density = line-art or highly detailed photograph. Low edge density = plain background or out-of-focus photo. The implementation skips every other pixel (sampling at stride 2) for speed, then computes the luminance gradient to the right and downward neighbors (a simplified one-sided Sobel operator).

Classification Decision Tree

Category	Conditions	Routed to
seal	warm + square-ish + photo-like + edge>0.08	Seals, Unicorn motif, Manufacture
script	line-art: light bg + high edge + monochrome	Script, Fish sign, Parpola
map	dominant hue = green or brown-green	Urban planning, City comparisons
artifact	warm + photo-like (non-square)	Figurines, Priest-King, Dancing Girl
illustration	light bg + colorSpread < 35	Overview, Chronology
site-photo	photo-like (fallback)	Urban planning, Great Bath

8. Gemini Nano — Free On-Device Intelligence

Gemini Nano is a compact language model distributed as part of Google Chrome itself. Unlike GPT-4, Claude, or Gemini Pro, it runs entirely on the user's own hardware — on their CPU or integrated GPU. No tokens are consumed. No API key is required. No data is transmitted. It is the closest thing to 'free AI' that currently exists.

Availability	Chrome 127+ with Gemini Nano flag enabled in <code>chrome://flags/#optimization-guide-on-device-model</code>
API surface	<code>window.LanguageModel</code> (new) or <code>window.ai.languageModel</code> (legacy) — both handled by <code>gemini-nano.js</code>
Model size	~1.8 GB — downloaded once by Chrome, stored locally, reused across all sites
Cost	Absolutely zero — uses no cloud, no tokens, no billing
Privacy	100% — text never leaves the device; processed entirely in the browser
Fallback	If unavailable, the KB answer is shown alone; the site is fully functional without it
System prompt	Hard-coded Indus domain lock — same scholarly rules as the KB
Parameters	Temperature: 0.4 (focused), TopK: 3 (very constrained)

The integration design is deliberately conservative: Gemini Nano only adds a supplementary paragraph to an already-correct KB answer. It is never the primary source. This prevents hallucinations from reaching the user without a safety net — the KB answer is always shown first, and if the Nano output contradicts it, the user has the KB content to compare against.

9. 3D Graphics — Two WebGL Scenes

WebGL is the browser's 3D graphics API, the same technology used for browser-based games. Three.js is a JavaScript library that wraps WebGL's complex low-level commands into a clean, readable scene-graph API. The project uses Three.js 0.160 loaded via CDN (no local install, no build step).

Why Two Separate Scenes?

Each WebGL renderer is independent — its own canvas, camera, scene graph, and animation loop. This allows the hero seal to have a completely different visual style (dark, intimate, close-up) from the trade routes globe (aerial, expansive, geopolitical). Both use IntersectionObserver to pause rendering when scrolled off-screen, saving GPU power on mobile devices.

Three.js features used across both scenes:

- PerspectiveCamera with orbit
 - WebGLRenderer (antialias, alpha)
 - ACESFilmic tone mapping
 - FogExp2 atmospheric fog
 - AmbientLight + DirectionalLight
 - PointLight fill lights
 - MeshStandardMaterial (PBR)
 - MeshBasicMaterial (flat shading)
 - ExtrudeGeometry (seal body, motif)
- TubeGeometry (trade route arcs)
 - QuadraticBezierCurve3 (arc paths)
 - CircleGeometry (world disc)
 - RingGeometry (halo rings, grid)
 - SphereGeometry (sparks, nodes)
 - CylinderGeometry (city pillars)
 - BufferGeometry + BufferAttribute
 - PointsMaterial + Points (dust)
 - AdditiveBlending (glow effects)

Renderer Settings

antialias: true	Smooths jagged edges — more GPU cost but much better visual quality
alpha: true	Transparent background — the canvas blends into the page background
outputColorSpace	THREE.SRGBColorSpace — correct gamma for display
toneMapping	ACESFilmicToneMapping — filmic HDR look, preserves highlight detail
pixelRatio	Math.min(devicePixelRatio, 2) — sharp on HiDPI but caps at 2x for performance

10. Design System & Visual Identity

The visual identity of Indus Valley AI is built around a single concept: *ancient meets futuristic*. The color palette is drawn from Harappan artifacts — fired steatite (warm gray), terracotta (brick red), and carnelian (deep amber-gold). The typography pairs a classical serif (Cormorant Garamond — used in museum publications) with a modern geometric sans (Inter) and a technical monospace (JetBrains Mono).

UI Animation Inventory

Custom cursor	requestAnimationFrame lerp at 0.30 (dot) and 0.12 (ring)
Spotlight glow	CSS radial-gradient tracking mouse via custom properties
Magnetic buttons	transform: translate() at 25% pointer offset magnitude
Reveal on scroll	opacity + scale transition, 800ms cubic-bezier(0.16,1,0.3,1)
Scroll spine fill	scaleY() interpolated to scroll percentage
Ticker text	@keyframes scrollLeft — infinite horizontal marquee
Seal gallery cards	backdrop-filter: blur() + translateY on hover
Halo rings (3D)	Three.js scale pulsing via sin(t) in animation loop
Trade sparks (3D)	QuadraticBezierCurve3.getPoint(t) travel, t cycles 0→1
Gold dust (3D)	Y-position drift, reset on reaching max height
Drop zone border	@keyframes pulsing dashed gold border on file drag
Typing indicator	Three bouncing dots (CSS @keyframes, staggered delay)

Responsive Design

The site is fully responsive. Below 768px: the scroll spine hides, the 3D canvas height reduces, navigation collapses into a burger menu, grid layouts switch to single-column, and the custom cursor is disabled (pointer:coarse media query). The site is usable on a phone, tablet, and desktop without any layout breakage.

11. Tech Stack Summary Table

Technology	Version / Source	Used For	Cost
HTML5	Semantic framework — no	Page structure, 12 sections	Free
CSS3	Vanilla preprocessor — no	Full design system, animations	Free
JavaScript (ES2022)	Vanilla IIFE + modules	All UI, AI, vision logic	Free
Three.js	0.160 via importmap CDN	Two real-time 3D WebGL scenes	Free
Gemini Nano	window.LanguageModel — Chrome built-in	On-device LLM enrichment	Free
Canvas 2D API	Browser built-in	Image pixel analysis (vision.js)	Free
IntersectionObserver	Browser built-in API	Scroll reveals, 3D pause	Free
Cormorant Garamond	Google Fonts CDN	Display / heading typeface	Free
Inter	Google Fonts CDN	Body typeface	Free
JetBrains Mono	Google Fonts CDN	Code / technical typeface	Free
Python http.server	Python 3 stdlib	Local development server	Free
ReportLab	4.4 — pip install	This PDF report generation	Free (OSS)

12. What We Built vs What Others Do

Most digital humanities projects fall into one of two categories: (a) simple static websites with text and images but no interactivity, or (b) complex platforms requiring cloud infrastructure and ongoing maintenance costs. Indus Valley AI occupies a unique third position.

Wikipedia

Comprehensive text reference. No AI, no 3D, no domain restriction. Anyone can edit, risking misinformation. No image analysis.

Google Arts & Culture

Beautiful visuals, museum partnerships. No AI chat. No domain-specific scholarly depth. Requires internet for every asset.

ChatGPT / Claude

General-purpose AI — powerful but unfocused and expensive to run. Will hallucinate about Indus topics. Not specialized, not offline.

Academic websites (e.g. harappa.com)

Authoritative scholarship. Static HTML, text-only. No AI, no 3D, no interaction layer.

Indus Valley AI (this project)

Domain-locked AI with 49 curated scholarly topics. Two real-time 3D scenes. On-device LLM enrichment. Image analysis. Zero server cost. Academic honesty built in. Works offline after first load.

13. Future Directions

Service Worker + Offline Cache

Registering a Service Worker would cache all assets after the first visit, making the site fully functional with no internet connection — a true offline PWA.

WebXR / Augmented Reality

Three.js supports WebXR. The 3D seal could be viewed in augmented reality on a mobile phone camera, overlaying the virtual seal over a real desk surface.

Expanded Knowledge Base

The KB currently covers 49 topics. A future version could expand to 150+ topics covering every major excavated site, every known motif category, and every published decipherment hypothesis in detail.

Vector Similarity Search

Instead of keyword scoring, embedding the KB topics as semantic vectors and using cosine similarity would allow the AI to match questions that use different words to describe the same concept.

Multi-language Support

Adding Hindi, Telugu, and Tamil interfaces would make the platform accessible to regional audiences in India, particularly for school and university education.

Peer-Reviewed Knowledge Validation

Partnering with archaeologists at DECCAN College, ASI, or IIT Gandhinagar to formally validate the KB content would make the platform citable as a scholarly source.

14. Conclusion

Indus Valley AI demonstrates that a sophisticated AI-powered educational platform does not require cloud infrastructure, paid APIs, or a backend server. By combining a hand-curated scholarly knowledge base, browser-native machine learning (Gemini Nano), client-side computer vision, and real-time 3D WebGL graphics — all within a single static website — the project achieves what would typically require a team of engineers and a monthly cloud bill.

Every design decision prioritized three values: academic honesty (the AI only answers what it knows, cites scholars, and flags uncertainty), privacy (no data ever leaves the user's device), and accessibility (free to use, free to host, works on any device).

The Indus Valley Civilization is one of the most important and least understood chapters of human history. Its script remains undeciphered. Its religion is debated. Its cities show a level of urban planning that rivals ancient Rome — 4,000 years earlier. A platform that makes this knowledge accessible, beautiful, and interactive is a small but meaningful contribution to that scholarship.

Project: Indus Valley AI | **Lead:** Mohammed Majeed Khan | **Organization:** AI Hub, Mahindra University | **Date:** April 2026

This report was generated programmatically using Python ReportLab. All code, content, and 3D scenes were purpose-built for this project.